



Control Flow in Java

Type	Lecture
Date	@January 3, 2022
Lecture #	3
Lecture URL	https://youtu.be/ORHDEourhdo
Notion URL	https://21f1003586.notion.site/Control-Flow-in-Java-3a68f0e8e0fb49a9babfea919410dc68
Week #	2

Control flow

- Program layout
 - Statements end with semi-colon ;
 - Blocks of statements delimited by braces
- Conditional execution
 - `if (condition) { ... } else { ... }`
- Conditional Loop
 - `while (condition) { ... }`
 - `do { ... } while (condition);`
- Iteration
 - Two kinds of `for`
- Multiway branching — `switch`

Conditional Execution

- `if (c) { ... } else { ... }`
 - `else` is optional
 - Condition must be in parentheses
 - If body is a single statement, braces are not needed
- No `elif`, unlike python
 - Indentation is not forced

- Just align `else if`
- Nested `if` is a single statement, no separate braces required
- No surprises
- Aside: No `def` for function definition

```
public class MyClass {
    ...
    public static int sign(int v) {
        if (v < 0)
            return(-1);
        else if (v > 0)
            return(1);
        else
            return(0);
    }
}
```

Conditional Loops

- `while (c) { ... }`
 - Condition must be in parentheses
 - If body is a single statement, braces are not needed

```
public class MyClass {
    ...
    public static int sumupto(int n) {
        int sum = 0;
        while (n > 0) {
            sum += n;
            n--;
        }
        return(sum);
    }
}
```

- `do { ... } while (c);`
 - Condition is checked at the end of the loop
 - At one iteration, even if the condition is false
 - Useful for interactive user-input

```
do {
    read input;
} while (input-condition);
```

```
public class MyClass {
    ...
    public static int sumupto(int n) {
        int sum = 0;
        int i = 0;
        do {
            sum += i;
            i++;
        } while (i <= n);
        return(sum);
    }
}
```

Iteration

- `for` loop is inherited from C language
- `for (init; cond; upd) { ... }`
 - `init` is initialization
 - `cond` is terminating condition
 - `upd` is loop update

- Intended use is

```
for (i = 0; i < n; ++i) { ... }
```

- Completely equivalent to

```
i = 0;
while (i < n) {
    i++;
}
```

- However, not a good style to write `for` instead of `while`
- Can define loop variable within loop
 - The scope of `i` is local to the loop
 - An instance of more general local scoping allowed in Java

Sample code

```
public class MyClass {
    ...
    public static int sumarray(int[] a) {
        int sum = 0;
        int n = a.length;
        int i;

        for(i = 0; i < n; ++i) {
            sum += a[i];
        }

        return(sum);
    }
}
```

Iterating over elements directly

- Java later introduced a `for` in the style of Python

```
for x in l:
    do something with x
```

- Again `for`, different syntax

```
for (type x : a) {
    do something with x;
}
```

- It appears that loop variable must be declared in local scope for this version of `for`

Sample code

```
public class MyClass {
    ...
    public static int sumarray(int[] a) {
        int sum = 0;
        int n = a.length;

        for(int v : a) {
            sum += v;
        }

        return(sum);
    }
}
```

Multiway branching

- `switch` selects between different options
- The default behaviour of `switch` is to "fall through" from one case to the next
 - Need to explicitly `break` out of switch

- `break` available for loops as well
- Options have to be constants
 - Cannot use conditional expressions
- Aside: here return type is `void`
 - Non-void return type requires an appropriate return value

Sample code

```
public static void printsign(int v) {  
    switch(v) {  
        case -1: {  
            System.out.println("Negative");  
            break;  
        }  
        case 1: {  
            System.out.println("Positive");  
            break;  
        }  
        case 0: {  
            System.out.println("Zero");  
            break;  
        }  
    }  
}
```

Summary

- Program layout: semi-colons, braces
- Conditional execution: `if, else`
- Conditional loops: `while, do-while`
- Iteration: two kinds of `for`
 - Local declaration of loop variable
- Multiway branching: `switch`
 - `break` to avoid falling through